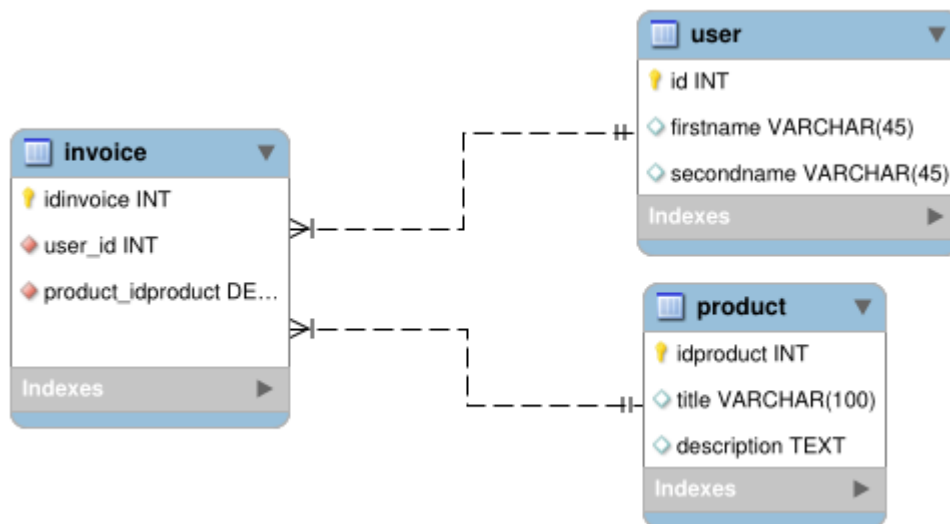


Лабораторная 2

Задание 1:

В ходе выполнения задания была спроектирована EER-диаграмма (модель), состоящая из трех таблиц: user, product и invoice. Таблицы user и product являются родительскими (справочниками), а таблица invoice — дочерней, связанной с ними отношением "один ко многим".

1. Схема EER-диаграммы в виде изображения:



2. SQL-скрипт для создания базы данных:

Полный SQL-скрипт для создания модели был сгенерирован с помощью Forward Engineer, загружен на сервис Github Gist и доступен по [ссылке](#)



3. Фрагмент SQL-скрипта: создание таблицы invoice:

-- Table mydb.invoice

```
CREATE TABLE IF NOT EXISTS mydb.invoice ( idinvoice INT NOT NULL
AUTO_INCREMENT, user_id INT NOT NULL, product_idproduct INT NOT NULL, cost
DECIMAL(10,2) NOT NULL, PRIMARY KEY (idinvoice), INDEX fk_invoice_user_idx
(user_id ASC) VISIBLE, INDEX fk_invoice_product1_idx (product_idproduct ASC)
VISIBLE, CONSTRAINT fk_invoice_user FOREIGN KEY (user_id) REFERENCES mydb.user
(id) ON DELETE CASCADE ON UPDATE CASCADE, CONSTRAINT fk_invoice_product1
FOREIGN KEY (product_idproduct) REFERENCES mydb.product (idproduct) ON DELETE
CASCADE ON UPDATE CASCADE) ENGINE = InnoDB;
```

```
-- -----
-- Table `mydb`.`invoice`
-- -----

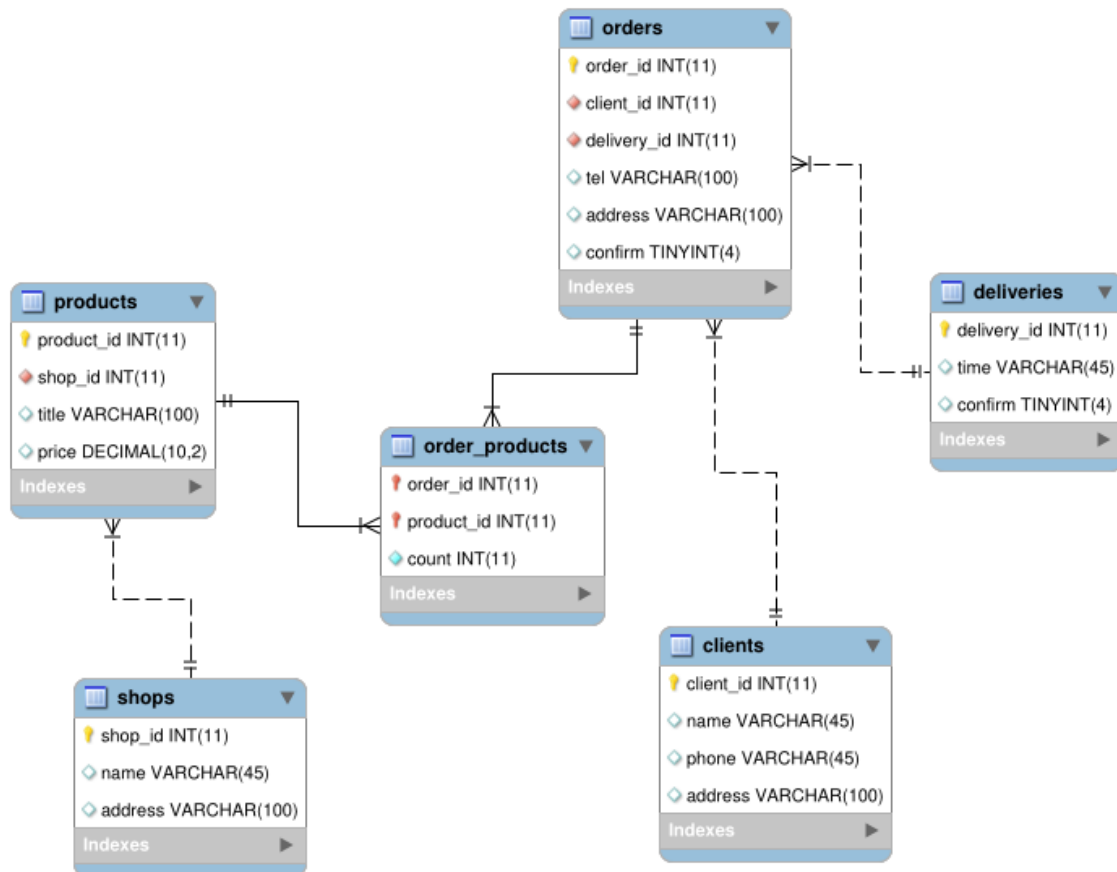
CREATE TABLE IF NOT EXISTS `mydb`.`invoice` (
  `idinvoice` INT NOT NULL AUTO_INCREMENT,
  `user_id` INT NOT NULL,
  `product_idproduct` INT NOT NULL,
  `cost` DECIMAL(10,2) NOT NULL,
  PRIMARY KEY (`idinvoice`),
  INDEX `fk_invoice_user_idx` (`user_id` ASC) VISIBLE,
  INDEX `fk_invoice_product1_idx` (`product_idproduct` ASC) VISIBLE,
  CONSTRAINT `fk_invoice_user`
    FOREIGN KEY (`user_id`)
      REFERENCES `mydb`.`user` (`id`)
      ON DELETE CASCADE
      ON UPDATE CASCADE,
  CONSTRAINT `fk_invoice_product1`
    FOREIGN KEY (`product_idproduct`)
      REFERENCES `mydb`.`product` (`idproduct`)
      ON DELETE CASCADE
      ON UPDATE CASCADE)
ENGINE = InnoDB;
```

Задание 2:

В рамках второго задания была спроектирована собственная EER-диаграмма для базы данных службы доставки на основе текстового описания и изображений из

статьи на Habr. Модель включает в себя 6 таблиц: shops, products, clients, deliveries, orders и связующую таблицу order_products, которые отражают все необходимые сущности и их взаимосвязи.

1. Схема EER-диаграммы в виде изображения:



2. SQL-скрипт для создания базы данных:

[ССЫЛКА](#)



3. Фрагмент SQL-скрипта: создание таблицы Orders:

Фрагмент SQL-кода, отвечающий за создание и настройку ключевой таблицы Orders со всеми ее полями и внешними ключами:

```

----- Table `delivery_db`.`orders` -----
----- CREATE TABLE IF NOT EXISTS
`delivery_db`.`orders` ( `order_id` INT NOT NULL AUTO_INCREMENT, `client_id` INT
NOT NULL, `delivery_id` INT NOT NULL, `tel` VARCHAR(100) NULL, `address`
VARCHAR(100) NULL, `confirm` TINYINT NULL, PRIMARY KEY (`order_id`), INDEX
`fk_orders_clients_idx` (`client_id` ASC) VISIBLE, INDEX `fk_orders_deliveries_idx`
(`delivery_id` ASC) VISIBLE, CONSTRAINT `fk_orders_clients` FOREIGN KEY
(`client_id`) REFERENCES `delivery_db`.`clients` (`client_id`), CONSTRAINT
`fk_orders_deliveries` FOREIGN KEY (`delivery_id`) REFERENCES
`delivery_db`.`deliveries` (`delivery_id`)) ENGINE = InnoDB;

```

```

-----
-- Table `delivery_db`.`orders`
-----
CREATE TABLE IF NOT EXISTS `delivery_db`.`orders` (
  `order_id` INT NOT NULL AUTO_INCREMENT,
  `client_id` INT NOT NULL,
  `delivery_id` INT NOT NULL,
  `tel` VARCHAR(100) NULL,
  `address` VARCHAR(100) NULL,
  `confirm` TINYINT NULL,
  PRIMARY KEY (`order_id`),
  INDEX `fk_orders_clients_idx` (`client_id` ASC) VISIBLE,
  INDEX `fk_orders_deliveries_idx` (`delivery_id` ASC) VISIBLE,
  CONSTRAINT `fk_orders_clients`
    FOREIGN KEY (`client_id`)
      REFERENCES `delivery_db`.`clients` (`client_id`),
  CONSTRAINT `fk_orders_deliveries`
    FOREIGN KEY (`delivery_id`)
      REFERENCES `delivery_db`.`deliveries` (`delivery_id`))
ENGINE = InnoDB;

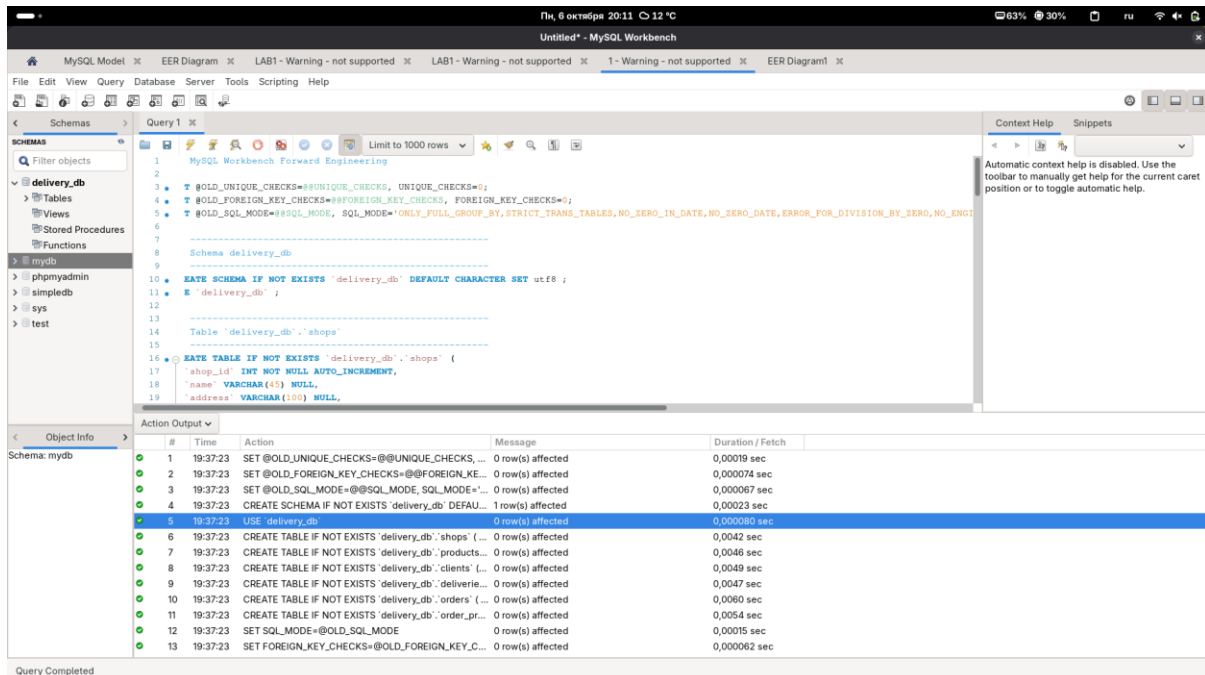
```

Задание 3

Для создания физической базы данных на сервере была выполнена операция Database -> Forward Engineer. В качестве входных данных был использован SQL-скрипт, сгенерированный на основе модели из Задания 2.

Мастер подключения успешно соединился с локальным сервером, после чего выполнил весь SQL-скрипт. В результате была создана новая база данных `delivery_db` со всеми спроектированными таблицами, индексами и внешними ключами.

Скриншот, подтверждающий успешное выполнение процесса:



Процесс завершился без ошибок, что подтверждает корректность спроектированной модели и сгенерированного SQL-кода.

Задание 4

В рамках данного задания спроектированная база данных `delivery_db` была наполнена тестовыми данными для проверки ее функциональности. Также была произведена проверка работы ограничений целостности при попытке удаления связанных записей.

1. Наполнение базы данных и добавление атрибутов:

Все таблицы (`shops`, `clients`, `deliveries`, `products`, `orders`, `order_products`) были успешно заполнены несколькими строками тестовых данных с помощью SQL-запросов `INSERT`.

Дополнительно, в качестве примера модификации таблиц, в таблицу `clients` был добавлен новый атрибут `email` для хранения электронной почты клиента.

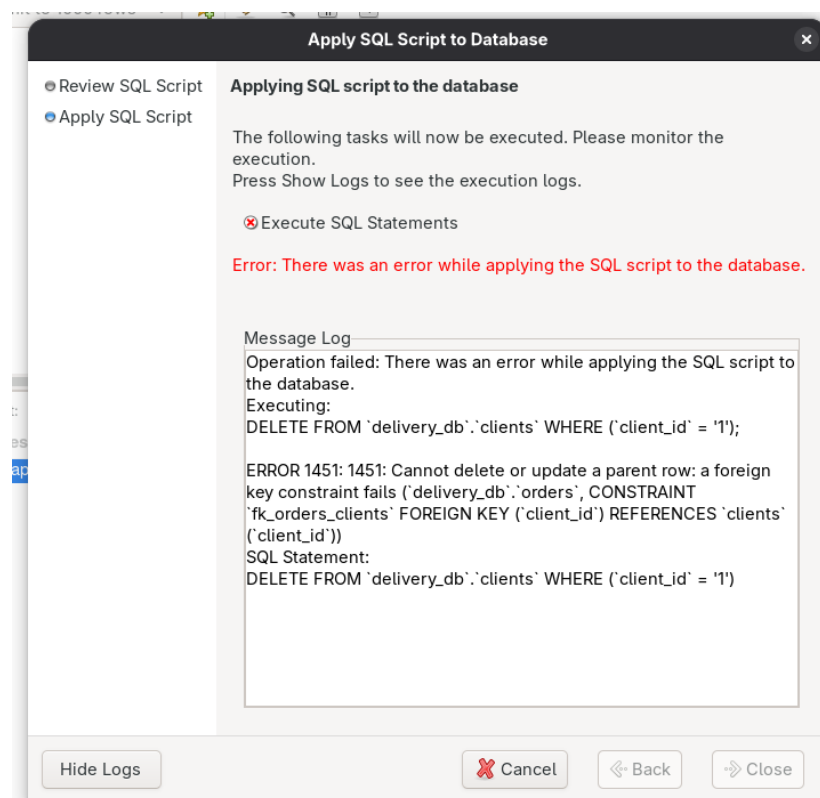
SQL-запрос для добавления атрибута:

```
ALTER TABLE `delivery_db`.`clients`  
ADD COLUMN `email` VARCHAR(100) NULL AFTER `phone`;
```

2. Проверка ограничений целостности при удалении:

Была произведена попытка удалить запись из родительской таблицы clients (клиент "Иван Иванов" с client_id = 1), на которую существует ссылка из дочерней таблицы orders (заказ с order_id = 1).

Результат выполнения:



Operation failed: There was an error while applying the SQL script to the database. Executing: DELETE FROM `delivery\_db`.`clients` WHERE (`client\_id` = '1'); ERROR 1451: Cannot delete or update a parent row: a foreign key constraint fails (`delivery\_db`.`orders`, CONSTRAINT `fk\_orders\_clients` FOREIGN KEY (`client\_id`) REFERENCES `clients` (`client\_id`)) SQL Statement: DELETE FROM `delivery\_db`.`clients` WHERE (`client\_id` = '1')

Анализ результата:

Удаление записи не удалось, так как сработало ограничение внешнего ключа (` FOREIGN KEY constraint `). Поскольку для связи между таблицами ` clients ` и ` orders ` не было установлено правило каскадного удаления (` ON DELETE CASCADE `), СУБД по умолчанию использует ограничивающее поведение (` RESTRICT `), чтобы предотвратить нарушение ссылочной целостности.

Таким образом, система не позволила удалить "родительскую" запись (клиента), пока существуют связанные с ней "дочерние" записи (заказы). Это подтверждает корректную работу спроектированных ограничений и обеспечивает защиту данных от появления "осиротевших" записей.